Lecture 5: Model-Free Prediction

Rigorous Mathematical Notes

Based on the UCL Reinforcement Learning Course, 2021
Lecturer: Hado van Hasselt

Contents

1 Setting and Preliminaries

We operate within the framework of a Markov Decision Process (MDP), which may be *unknown* to the agent.

Definition 1.1 (Markov Decision Process)**.** A (finite, discounted) MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$ where $\mathcal{S}$ is a finite state space, $\mathcal{A}$ is a finite action space, $p(s' \mid s, a)$ is the transition kernel, $r(s, a)$ is the expected reward function, and $\gamma \in [0, 1)$ is the discount factor.

Definition 1.2 (Policy)**.** A (stationary, stochastic) policy is a mapping $\pi : \mathcal{S} \to \Delta(\mathcal{A})$, where $\Delta(\mathcal{A})$ denotes the probability simplex over $\mathcal{A}$. We write $\pi(a \mid s)$ for the probability of selecting action a in state s.

Definition 1.3 (Return)**.** Given a trajectory $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, \dots)$ with episode termination at time $T > t$, the *return* from time t is

$$G_t \;=\; \sum_{k=0}^{T-t-1} \gamma^k \, R_{t+k+1} \;=\; R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T.$$

Definition 1.4 (State-Value Function)**.** Under policy π, the state-value function is

$$v_\pi(s) \;=\; \mathbb{E}_\pi[G_t \mid S_t = s], \qquad \forall\, s \in \mathcal{S}.$$

Definition 1.5 (Action-Value Function)**.** Under policy π, the action-value function is

$$q_\pi(s, a) \;=\; \mathbb{E}_\pi[G_t \mid S_t = s,\, A_t = a], \qquad \forall\, (s, a) \in \mathcal{S} \times \mathcal{A}.$$

Definition 1.6 (Model-Free Prediction)**.** The *model-free prediction* problem consists of estimating v_π (or q_π) given only sample trajectories generated under policy π, without knowledge of the transition kernel p or the reward function r.

Remark 1.7 (Planning vs. Learning)**.** In lectures 3–4, the MDP was assumed known and v_π was computed via dynamic programming (*planning*). In this lecture the MDP is *unknown*; the agent must learn from sampled experience.

2 Monte Carlo Algorithms

2.1 Monte Carlo for Bandits

Definition 2.1 (Multi-Armed Bandit)**.** A multi-armed bandit is a single-state MDP $(\{s_0\}, \mathcal{A}, p, r, \gamma)$ in which $|\mathcal{S}| = 1$, so actions have no long-term consequences. The true action value is

$$q(a) \;=\; \mathbb{E}[R_{t+1} \mid A_t = a].$$

Definition 2.2 (Sample-Average Estimator)**.** After t time steps, the sample-average estimator of $q(a)$ is

$$q_t(a) \;=\; \frac{\sum_{i=0}^{t-1} \mathbb{1}(A_i = a)\, R_{i+1}}{\sum_{i=0}^{t-1} \mathbb{1}(A_i = a)} \;=\; \frac{\sum_{i=0}^{t-1} \mathbb{1}(A_i = a)\, R_{i+1}}{N_t(a)},$$

where $N_t(a) = \sum_{i=0}^{t-1} \mathbb{1}(A_i = a)$ is the number of times action a has been selected.

Proposition 2.3 (Incremental Update Form)**.** *The sample-average estimator satisfies the incremental update*

$$q_{t+1}(A_t) \;=\; q_t(A_t) + \alpha_t (R_{t+1} - q_t(A_t)), \qquad q_{t+1}(a) = q_t(a) \;\; \forall\, a \neq A_t,$$

with step size $\alpha_t = 1/N_t(A_t)$.

Proof. Write $N = N_t(A_t)$. Then

$$q_{t+1}(A_t) = \frac{1}{N+1}\left(\sum_{i:\, A_i = A_t} R_{i+1}\right) = \frac{1}{N+1}(N\, q_t(A_t) + R_{t+1}) = q_t(A_t) + \frac{1}{N+1}(R_{t+1} - q_t(A_t))\,.$$

□

2.2 Contextual Bandits

Definition 2.4 (Contextual Bandit)**.** A contextual bandit extends the multi-armed bandit to multiple states: episodes are single-step, and actions do not affect state transitions. The target quantity is

$$q(s,a) \;=\; \mathbb{E}[R_{t+1} \mid S_t = s,\, A_t = a]\,.$$

2.3 Monte Carlo Policy Evaluation in Sequential MDPs

Definition 2.5 (Episode)**.** An *episode* under policy π is a trajectory

$$S_1, A_1, R_2, S_2, A_2, R_3, \ldots, S_T,$$

where $A_t \sim \pi(\cdot \mid S_t)$ and $S_{t+1}, R_{t+1} \sim p(\cdot \mid S_t, A_t)$.

Algorithm 2.6 (Monte Carlo Policy Evaluation)**.** Given a set of episodes sampled under π:

1. For each episode and each time step t, compute the return G_t.
2. For each state s, update
$$v(s) \;\leftarrow\; v(s) + \alpha(G_t - v(s))\,,$$
for every visit t where $S_t = s$ (every-visit MC).

Theorem 2.7 (Unbiasedness of MC Returns)**.** *The Monte Carlo return G_t is an unbiased estimator of $v_\pi(S_t)$:*

$$\mathbb{E}_\pi[G_t \mid S_t = s] = v_\pi(s).$$

Proof. Immediate from the definition $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$. □

Remark 2.8 (Disadvantages of MC)**.** MC methods require complete episodes (terminating environments) and can suffer from high variance since G_t depends on the entire future trajectory. Learning is slow for long episodes because updates are deferred until episode termination.

3 Function Approximation

3.1 General Function Approximation

Definition 3.1 (Value Function Approximation)**.** Given a parameter vector $\mathbf{w} \in \mathbb{R}^d$, we approximate the true value function via a parametric model:

$$v_{\mathbf{w}}(s) \;\approx\; v_\pi(s), \qquad q_{\mathbf{w}}(s,a) \;\approx\; q_\pi(s,a).$$

The parameters $\mathbf{w}$ are learned from experience using, e.g., MC or TD methods.

Definition 3.2 (Agent State)**.** When the environment state is not fully observable, the agent maintains an *agent state*:

$$S_t \;=\; u_\omega(S_{t-1}, A_{t-1}, O_t),$$

where $\omega \in \mathbb{R}^n$ are parameters of the state-update function and O_t is the observation. In the simplest case $S_t = O_t$.

3.2 Linear Function Approximation

Definition 3.3 (Feature Vector)**.** A *feature map* is a fixed mapping $\mathbf{x} : \mathcal{S} \to \mathbb{R}^m$. We write $\mathbf{x}_t = \mathbf{x}(S_t)$.

Definition 3.4 (Linear Value Function)**.** The *linear value function approximation* is

$$v_{\mathbf{w}}(s) \;=\; \mathbf{w}^{\mathsf{T}}\mathbf{x}(s) \;=\; \sum_{j=1}^{m} w_j\, x_j(s).$$

Definition 3.5 (Mean Squared Value Error)**.** Under a state distribution d over $\mathcal{S}$, the loss is

$$L(\mathbf{w}) \;=\; \mathbb{E}_{S\sim d}\Big[\Big(v_\pi(S) - \mathbf{w}^{\mathsf{T}}\mathbf{x}(S)\Big)^2\Big].$$

Theorem 3.6 (Convergence of Linear SGD)**.** *The loss $L(\mathbf{w})$ is convex and quadratic in $\mathbf{w}$. Stochastic gradient descent with the update*

$$\Delta\mathbf{w} \;=\; \alpha(v_\pi(S_t) - v_{\mathbf{w}}(S_t))\,\mathbf{x}_t,$$

converges to the global minimum under standard step-size conditions ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$).

Proof sketch. We have $\nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) = \mathbf{x}_t$. The negative stochastic gradient of L at $\mathbf{w}$ yields

$$-\tfrac{1}{2}\nabla_{\mathbf{w}}(v_\pi(S_t) - \mathbf{w}^{\mathsf{T}}\mathbf{x}_t)^2 = (v_\pi(S_t) - \mathbf{w}^{\mathsf{T}}\mathbf{x}_t)\,\mathbf{x}_t.$$

Since L is strongly convex (assuming $\mathbb{E}_d[\mathbf{x}\mathbf{x}^{\mathsf{T}}]$ is positive definite), standard SGD convergence results apply. □

Proposition 3.7 (Table Lookup as a Special Case)**.** *Let $\mathcal{S} = \{s_1, \dots, s_n\}$. Using one-hot features*

$$\mathbf{x}(s) = \big(\mathbb{1}(s = s_1), \dots, \mathbb{1}(s = s_n)\big)^{\mathsf{T}} \in \mathbb{R}^n,$$

we recover the tabular representation: $v_{\mathbf{w}}(s_i) = w_i$ for all $i \in \{1, \dots, n\}$.

Proof. $v_{\mathbf{w}}(s_i) = \mathbf{w}^{\mathsf{T}}\mathbf{x}(s_i) = \sum_{j=1}^{n} w_j \mathbb{1}(s_i = s_j) = w_i$. □

3.3 SGD for Contextual Bandits with Function Approximation

Definition 3.8 (Regression Loss for Contextual Bandits)**.** For a parametric action-value function $q_{\mathbf{w}}(s, a)$, the regression loss is

$$L(\mathbf{w}) \;=\; \tfrac{1}{2}\,\mathbb{E}\Big[(R_{t+1} - q_{\mathbf{w}}(S_t, A_t))^2\Big].$$

Proposition 3.9 (SGD Update — General Case)**.** *The stochastic gradient descent update for the regression loss is*

$$\mathbf{w}_{t+1} \;=\; \mathbf{w}_t + \alpha(R_{t+1} - q_{\mathbf{w}_t}(S_t, A_t))\,\nabla_{\mathbf{w}_t} q_{\mathbf{w}_t}(S_t, A_t).$$

Proof.

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = -\,\mathbb{E}[(R_{t+1} - q_{\mathbf{w}}(S_t, A_t))\,\nabla_{\mathbf{w}} q_{\mathbf{w}}(S_t, A_t)]\,.$$

A single-sample approximation of the negative gradient gives the stated update. □

Corollary 3.10 (Linear SGD Update)**.** *When $q_{\mathbf{w}}(s, a) = \mathbf{w}^{\mathsf{T}}\mathbf{x}(s, a)$, we have $\nabla_{\mathbf{w}} q_{\mathbf{w}} = \mathbf{x}(s, a)$ and the update simplifies to*

$$\mathbf{w}_{t+1} \;=\; \mathbf{w}_t + \alpha(R_{t+1} - q_{\mathbf{w}_t}(S_t, A_t))\,\mathbf{x}(S_t, A_t).$$

Remark 3.11. The update has the general form: step size × prediction error × gradient. In the linear case the gradient reduces to the feature vector.

4 Temporal-Difference Learning

4.1 Motivation from the Bellman Equation

Theorem 4.1 (Bellman Expectation Equation). *The value function v_π satisfies*

$$v_\pi(s) \;=\; \mathbb{E}_\pi[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s]\,.$$

Remark 4.2. Dynamic programming computes the full expectation. TD learning *samples* this expectation and uses the current value estimate v_t in place of v_π.

4.2 TD(0) Algorithm

Definition 4.3 (TD Target and TD Error). The *TD target* is $R_{t+1} + \gamma\, v_t(S_{t+1})$. The *TD error* is

$$\delta_t \;=\; R_{t+1} + \gamma\, v_t(S_{t+1}) - v_t(S_t).$$

Algorithm 4.4 (TD(0) for State Values — Tabular). Given experience under policy π, for each observed transition $(S_t, A_t, R_{t+1}, S_{t+1})$:

$$v_{t+1}(S_t) \;\leftarrow\; v_t(S_t) + \alpha\,\delta_t \;=\; v_t(S_t) + \alpha(R_{t+1} + \gamma\, v_t(S_{t+1}) - v_t(S_t))\,.$$

All other entries remain unchanged: $v_{t+1}(s) = v_t(s)$ for $s \neq S_t$.

4.3 SARSA: TD for Action Values

Algorithm 4.5 (SARSA — TD(0) for Action Values). Given a transition tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, update:

$$q_{t+1}(S_t, A_t) \;\leftarrow\; q_t(S_t, A_t) + \alpha(R_{t+1} + \gamma\, q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t))\,.$$

Remark 4.6. The name SARSA is a mnemonic for the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ used in each update.

4.4 Comparison of DP, MC, and TD Backups

Definition 4.7 (Backup Operations). The three fundamental backup operations are:

(i) **DP backup (full expectation, bootstrapping):**

$$v(S_t) \;\leftarrow\; \mathbb{E}_\pi[R_{t+1} + \gamma\, v(S_{t+1}) \mid S_t]\,.$$

(ii) **MC backup (sampling, no bootstrapping):**

$$v(S_t) \;\leftarrow\; v(S_t) + \alpha(G_t - v(S_t))\,.$$

(iii) **TD backup (sampling + bootstrapping):**

$$v(S_t) \;\leftarrow\; v(S_t) + \alpha(R_{t+1} + \gamma\, v(S_{t+1}) - v(S_t))\,.$$

Definition 4.8 (Bootstrapping and Sampling). An update *bootstraps* if it uses an estimate of the value function in the target. An update *samples* if it replaces an expectation with a single sample.

	Bootstraps	Samples
DP	Yes	No
MC	No	Yes
TD	Yes	Yes

4.5 Properties of TD Learning

Proposition 4.9 (Properties of TD(0))**.** *TD(0) has the following properties:*

(i) It is model-free*: no knowledge of p or r is required.*

(ii) It learns from incomplete *episodes (online, after every step).*

(iii) It applies to continuing *(non-terminating) tasks.*

(iv) It is independent of the temporal span of the prediction.

4.6 Bias–Variance Trade-off

Theorem 4.10 (Bias and Variance of MC vs. TD Targets)**.**

(i) The MC return G_t is an unbiased *estimate of $v_\pi(S_t)$.*

(ii) The TD target $R_{t+1} + \gamma\, v_t(S_{t+1})$ is a biased *estimate of $v_\pi(S_t)$ whenever $v_t \neq v_\pi$.*

(iii) The TD target generally has lower variance *than G_t, because G_t depends on the full sequence of stochastic actions, transitions, and rewards, while the TD target depends on only a single transition.*

Proof. (i) follows from Theorem 2.7.

(ii) $\mathbb{E}[R_{t+1} + \gamma v_t(S_{t+1}) \mid S_t = s] = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] + \gamma\, \mathbb{E}[v_t(S_{t+1}) - v_\pi(S_{t+1}) \mid S_t = s]$. The first term equals $v_\pi(s)$ by the Bellman equation; the second term is in general non-zero, constituting the bias.

(iii) Writing $G_t = R_{t+1} + \gamma G_{t+1}$, the variance accumulates over all future time steps, while the TD target replaces γG_{t+1} with $\gamma v_t(S_{t+1})$, which acts as a variance-reducing baseline (at the cost of bias). □

Remark 4.11 (Irreducible Bias of TD)**.** Under partial observability or with an insufficiently expressive function class, TD may exhibit irreducible bias. MC implicitly integrates over latent variables and is therefore unaffected by partial observability (though it may still suffer from high variance).

Theorem 4.12 (Tabular Convergence)**.** *In the tabular setting, both MC and TD(0) converge to v_π as the number of episodes tends to infinity, provided the step sizes satisfy $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$, and all states are visited infinitely often.*

4.7 Batch MC vs. Batch TD

Definition 4.13 (Batch Setting)**.** Given a fixed batch of K episodes

$$\{(S_1^k, A_1^k, R_2^k, \ldots, S_{T_k}^k)\}_{k=1}^K,$$

we repeatedly sample episodes from this batch and apply MC or TD(0) updates until convergence.

Theorem 4.14 (Batch MC Solution)**.** *Batch MC converges to the value function that minimises the mean-squared error of the observed returns:*

$$v_{\mathrm{MC}} \;=\; \arg\min_v \sum_{k=1}^{K} \sum_{t=1}^{T_k} \Big(G_t^k - v(S_t^k)\Big)^2 .$$

Theorem 4.15 (Batch TD Solution). *Batch TD(0) converges to the value function of the* maximum-likelihood MDP *estimated from the data. That is, TD solves the empirical MDP* $(\mathcal{S}, \mathcal{A}, \hat{p}, \hat{r}, \gamma)$ *where*

$$\hat{p}(s' \mid s, a) = \frac{\#(s, a, s')}{\#(s, a)}, \qquad \hat{r}(s, a) = \frac{\sum_{(s,a)\ visits} R}{\#(s, a)}.$$

Remark 4.16 (Markov Property Exploitation). TD exploits the Markov property by fitting the empirical MDP; this is advantageous in fully observable environments. MC does not assume the Markov property and can therefore be preferable in partially observable settings.

5 Multi-Step TD Methods

5.1 n-Step Returns

Definition 5.1 (n-Step Return). For $n \geq 1$, the *n-step return* from time t is

$$G_t^{(n)} \;=\; \sum_{k=0}^{n-1} \gamma^k \, R_{t+k+1} \;+\; \gamma^n \, v(S_{t+n}).$$

Remark 5.2 (Special Cases).

(i) $n = 1$: $G_t^{(1)} = R_{t+1} + \gamma \, v(S_{t+1})$ (TD(0) target).

(ii) $n = 2$: $G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 v(S_{t+2})$.

(iii) $n \to \infty$ (or $n = T - t$): $G_t^{(\infty)} = G_t$ (MC return).

Algorithm 5.3 (n-Step TD). The n-step TD update is

$$v(S_t) \;\leftarrow\; v(S_t) + \alpha\Big(G_t^{(n)} - v(S_t)\Big)\,.$$

Remark 5.4 (Bias–Variance Interpolation). Increasing n reduces bootstrapping bias but increases variance. In practice, intermediate values of n (e.g., $n \approx 10$) often yield the best performance.

6 Mixed Multi-Step Returns (λ-Returns)

6.1 The λ-Return

Definition 6.1 (λ-Return). For $\lambda \in [0, 1]$, the λ-return is defined as

$$G_t^{\lambda} \;=\; (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \, G_t^{(n)}.$$

Equivalently, it satisfies the recursion

$$G_t^{\lambda} \;=\; R_{t+1} + \gamma \Big[(1 - \lambda) \, v(S_{t+1}) + \lambda \, G_{t+1}^{\lambda}\Big]\,.$$

Proposition 6.2 (Weights Sum to One). *The weighting coefficients form a valid probability distribution:*

$$\sum_{n=1}^{\infty} (1 - \lambda) \, \lambda^{n-1} \;=\; 1.$$

Proof. This is a geometric series: $(1-\lambda)\sum_{n=0}^{\infty}\lambda^n = (1-\lambda)\cdot\frac{1}{1-\lambda} = 1$. □

Proposition 6.3 (Special Cases of λ-Return)**.**

(i) $\lambda = 0$*:* $G_t^{\lambda=0} = R_{t+1} + \gamma\, v(S_{t+1})$ *(TD(0) target).*

(ii) $\lambda = 1$*:* $G_t^{\lambda=1} = R_{t+1} + \gamma\, G_{t+1}$ *(MC return).*

Remark 6.4 (Effective Horizon)**.** The parameter λ controls the effective horizon of the update. Intuitively, $1/(1-\lambda)$ serves as an approximate horizon length; e.g., $\lambda = 0.9$ corresponds roughly to $n \approx 10$.

6.2 Benefits of Multi-Step and λ-Returns

Remark 6.5 (Benefits)**.** Multi-step and λ-returns combine the advantages of both TD and MC: bootstrapping reduces variance, while using longer returns mitigates bias from inaccurate value estimates. Empirically, intermediate values of λ (e.g., $\lambda = 0.9$) often achieve the best performance.

7 Eligibility Traces

7.1 Motivation

Remark 7.1. MC and multi-step returns are not independent of the temporal span of predictions: to update values in a long episode, one must wait until the required future rewards are observed. TD(0) updates immediately but propagates information slowly. Eligibility traces combine the best of both.

7.2 Decomposition of MC Error into TD Errors

Lemma 7.2 (Telescoping Decomposition)**.** *The MC error $G_t - v(S_t)$ decomposes as a discounted sum of TD errors:*

$$G_t - v(S_t) \;=\; \sum_{k=t}^{T-1} \gamma^{k-t}\,\delta_k,$$

where $\delta_k = R_{k+1} + \gamma\, v(S_{k+1}) - v(S_k)$.

Proof. By induction. Base case: at $t = T-1$, $G_{T-1} - v(S_{T-1}) = R_T - v(S_{T-1}) = \delta_{T-1}$ (with $v(S_T) = 0$).

Inductive step:

$$\begin{aligned}
G_t - v(S_t) &= R_{t+1} + \gamma G_{t+1} - v(S_t) \\
&= \big(R_{t+1} + \gamma v(S_{t+1}) - v(S_t)\big) + \gamma\big(G_{t+1} - v(S_{t+1})\big) \\
&= \delta_t + \gamma\big(G_{t+1} - v(S_{t+1})\big) \\
&= \delta_t + \gamma \sum_{k=t+1}^{T-1} \gamma^{k-t-1}\delta_k \\
&= \sum_{k=t}^{T-1} \gamma^{k-t}\delta_k.
\end{aligned}$$

□

7.3 Derivation of Eligibility Traces

Theorem 7.3 (Eligibility Trace Equivalence)**.** *Consider the batched episodic MC update with linear function approximation $v_{\mathbf{w}}(s) = \mathbf{w}^{\mathsf{T}}\mathbf{x}(s)$. The total weight update for an episode of length T is*

$$\Delta \mathbf{w} \;=\; \sum_{t=0}^{T-1} \alpha(G_t - v(S_t))\,\mathbf{x}_t \;=\; \sum_{t=0}^{T-1} \alpha\,\delta_t\,\mathbf{e}_t,$$

where the eligibility trace $\mathbf{e}_t \in \mathbb{R}^m$ *is defined by*

$$\mathbf{e}_t \;=\; \sum_{j=0}^{t} \gamma^{t-j}\,\mathbf{x}_j \;=\; \gamma\,\mathbf{e}_{t-1} + \mathbf{x}_t, \qquad \mathbf{e}_{-1} = \mathbf{0}.$$

Proof. Using Lemma 7.2:

$$\Delta \mathbf{w} = \sum_{t=0}^{T-1} \alpha \left(\sum_{k=t}^{T-1} \gamma^{k-t} \delta_k \right) \mathbf{x}_t = \sum_{k=0}^{T-1} \alpha\,\delta_k \underbrace{\sum_{t=0}^{k} \gamma^{k-t}\,\mathbf{x}_t}_{=\,\mathbf{e}_k} = \sum_{k=0}^{T-1} \alpha\,\delta_k\,\mathbf{e}_k.$$

The exchange of summation uses $\sum_{t=0}^{T-1}\sum_{k=t}^{T-1} f(t,k) = \sum_{k=0}^{T-1}\sum_{t=0}^{k} f(t,k)$.

The recursive form follows from:

$$\mathbf{e}_t = \sum_{j=0}^{t} \gamma^{t-j}\mathbf{x}_j = \gamma \sum_{j=0}^{t-1} \gamma^{t-1-j}\mathbf{x}_j + \mathbf{x}_t = \gamma\,\mathbf{e}_{t-1} + \mathbf{x}_t. \qquad \square$$

Remark 7.4 (Intuition)**.** The eligibility trace $\mathbf{e}_t$ is a fading memory of past feature vectors. When a TD error δ_t is observed, it is distributed to all previously visited states in proportion to their eligibility. This allows a single update $\Delta\mathbf{w}_t = \alpha\,\delta_t\,\mathbf{e}_t$ to propagate information to all past states simultaneously, without revisiting them.

Remark 7.5 (Tabular Special Case)**.** In the tabular setting, $\mathbf{x}_t$ is a one-hot vector. The eligibility trace $\mathbf{e}_t$ then tracks a decaying count of visits to each state, and the update $\alpha\,\delta_t\,\mathbf{e}_t$ applies the current TD error to all previously visited states with geometrically decaying weight.

7.4 λ-Returns and Eligibility Traces: TD(λ)

Theorem 7.6 (TD(λ) Eligibility Trace)**.** *The λ-return error decomposes analogously to Lemma 7.2:*

$$G_t^{\lambda} - v(S_t) \;=\; \sum_{k=0}^{T-t-1} (\lambda\gamma)^k\,\delta_{t+k}.$$

The corresponding batched episodic update is

$$\Delta \mathbf{w} \;=\; \sum_{t=0}^{T-1} \alpha\,\delta_t\,\mathbf{e}_t,$$

with the accumulating eligibility trace with decay $\gamma\lambda$*:*

$$\mathbf{e}_t \;=\; \gamma\lambda\,\mathbf{e}_{t-1} + \mathbf{x}_t, \qquad \mathbf{e}_{-1} = \mathbf{0}.$$

Proof. The proof proceeds identically to Theorem 7.3, with γ replaced by $\gamma\lambda$ in the telescoping decomposition. $\square$

Corollary 7.7 (Special Cases of TD(λ))**.**

(i) $\lambda = 0$*:* $\mathbf{e}_t = \mathbf{x}_t$*, recovering one-step TD(0).*

(ii) $\lambda = 1$*:* $\mathbf{e}_t = \gamma\,\mathbf{e}_{t-1} + \mathbf{x}_t$*, recovering (batched, offline) Monte Carlo.*

Algorithm 7.8 (TD(λ) — Online Forward View)**.** At each time step t:

1. Observe transition $(S_t, A_t, R_{t+1}, S_{t+1})$.
2. Compute TD error: $\delta_t = R_{t+1} + \gamma\, v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$.
3. Update eligibility trace: $\mathbf{e}_t = \gamma\lambda\,\mathbf{e}_{t-1} + \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)$.
4. Update parameters: $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\,\delta_t\,\mathbf{e}_t$.

Remark 7.9. The TD(λ) algorithm with accumulating traces is exact (equivalent to the forward-view λ-return algorithm) for batched offline episodic updates. For online (incremental) updating, similar but slightly different trace constructions exist that maintain exactness.

8 Summary of Algorithms

Algorithm	**Target**	**Update**	**Bootstrap**	**Sample**
DP	$\mathbb{E}[R_{t+1} + \gamma v(S_{t+1})]$	Full sweep	Yes	No
MC	G_t	$v(S_t) \leftarrow v(S_t) + \alpha(G_t - v(S_t))$	No	Yes
TD(0)	$R_{t+1} + \gamma v(S_{t+1})$	$v(S_t) \leftarrow v(S_t) + \alpha\,\delta_t$	Yes	Yes
n-step TD	$G_t^{(n)}$	$v(S_t) \leftarrow v(S_t) + \alpha(G_t^{(n)} - v(S_t))$	Yes	Yes
TD(λ)	G_t^{λ}	$\mathbf{w} \leftarrow \mathbf{w} + \alpha\,\delta_t\,\mathbf{e}_t$	Yes	Yes